

Part 1: The Phantom Bug

Part 1: 50,000,000 points across 4 threads, UNSYNCHRONIZED counter

Run 1: $\pi \approx 3.141528$ total_hits=39,269,106 time=6.52s

Run 2: $\pi \approx 3.141659$ total_hits=39,270,743 time=6.49s

Run 3: $\pi \approx 3.141194$ total_hits=39,264,923 time=6.46s

Run 4: $\pi \approx 3.141824$ total_hits=39,272,806 time=6.36s

Run 5: $\pi \approx 3.141394$ total_hits=39,267,430 time=6.40s

Summary (5 runs): ['3.141528', '3.141659', '3.141194', '3.141824', '3.141394']

Part 2: The Synchronization Trap

Part 2: 50,000,000 points

Locked, 4 threads: $\pi \approx 3.141510$ time = 11.09s

Single-threaded for loop: $\pi \approx 3.142024$ time = 5.65s

Slowdown factor (locked-threaded / single-threaded): 1.96x

Process finished with exit code 0

Part 3: OpenMP-Style Reduction

- Part 3: N = 100,000,000 points, reduction pattern, multiprocessing
- Detected CPU cores: 8
-
- T= 1 runtime= 11714.1 ms $\pi \approx 3.141623$
- T= 2 runtime= 6241.1 ms $\pi \approx 3.141628$
- T= 4 runtime= 4601.9 ms $\pi \approx 3.141365$
- T= 8 runtime= 3817.4 ms $\pi \approx 3.141430$
- T=16 runtime= 4063.0 ms $\pi \approx 3.141547$
- T=32 runtime= 5543.3 ms $\pi \approx 3.141488$

-
- | Threads (T) | Runtime (ms) | Speedup (T1/TN) | Efficiency (Speedup/T) |
- |---|---|---|---|
- | 1 | 11714.1 | 1.00x | 100% |
- | 2 | 6241.1 | 1.88x | 94% |
- | 4 | 4601.9 | 2.55x | 64% |
- | 8 | 3817.4 | 3.07x | 38% |
- | 16 | 4063.0 | 2.88x | 18% |
- | 32 | 5543.3 | 2.11x | 7% |
-

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (T/N T1)	Efficiency (T/Speedup)
1	• 11714.1	• 1.00x	• 100%
2	• 6241.1	• 1.88x	• 94%
4	• 4601.9	• 2.55x	• 64%
8	• 3817.4	• 3.07x	• 38%
16	• 4063.0	• 2.88x	• 18%

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

Once thread/process count exceeds the number of physical cores, you're no longer adding parallelism — you're adding **contention**. 16 workers on 8 cores means each core is now time-slicing 2 workers instead of running continuously, so you pay context-switch overhead (saving/restoring registers, cache flushes, scheduler bookkeeping) without gaining any actual extra concurrent computation. You may also be fighting hyperthreading limits (2 logical threads sharing one core's execution units, not doubling real throughput) and memory-bandwidth/cache contention across workers. The theoretical ceiling on 8 physical cores is $\sim 8\times$ speedup (minus the always-present serial portion, per Amdahl's Law); going past that with more workers plateaus or regresses runtime rather than improving it.

2. Why was the synchronized version in Part 2 slower than running on one single core?

Every single increment now requires acquiring a lock, which — even in the best case — costs far more than the ``+= 1`` it's protecting: it involves an atomic compare-and-swap or OS-level mutex operation, potential thread suspension if the lock is contended, and a memory barrier that forces the CPU to flush/invalidate cache lines rather than use cheap local cache. With 4 threads hammering one lock 50 million times, most of that time is spent waiting for the lock and synchronizing caches across cores, not doing the actual ``x*x + y*y`` math — so you end up doing **less** useful work per second than a single thread that never has to coordinate with anyone.

Share repo access with @sufyanism (GitHub/GitLab).